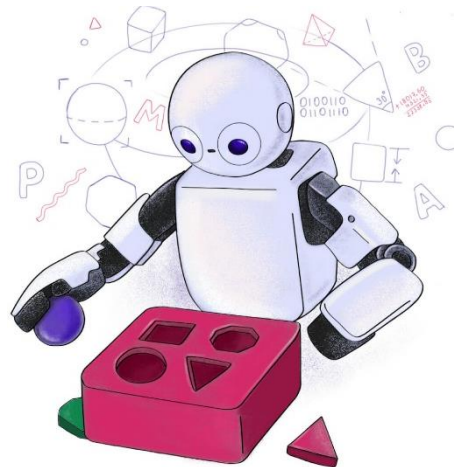
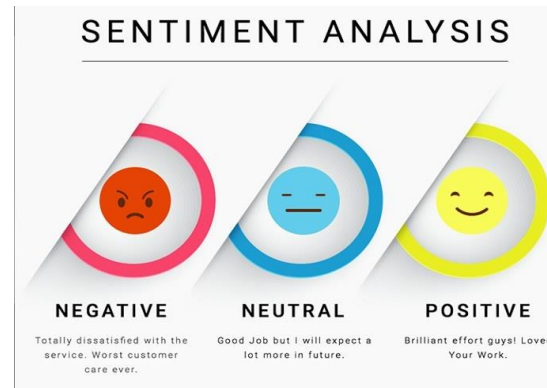
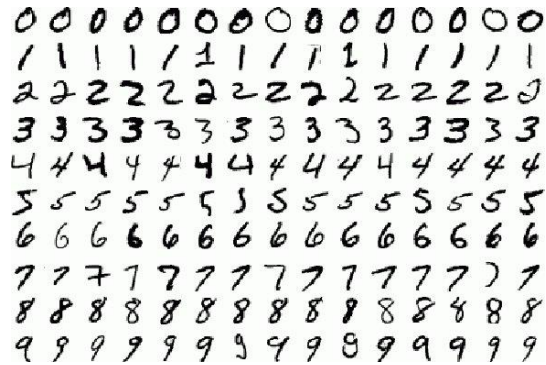


C24 – Inteligência Artificial: *Regressor Softmax*

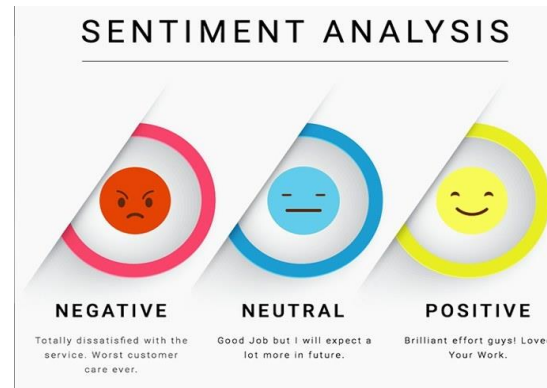
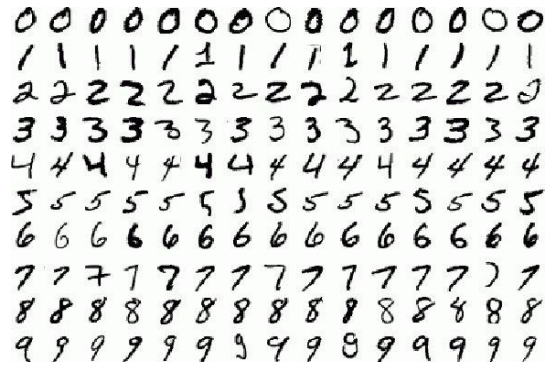


Casos multiclases



- Até agora, nós vimos como classificar utilizando a **regressão logística**.
 - Classificação binária.
- Mas e quando o problema possuir mais de 2 classes (i.e., $Q > 2$)?
- Por exemplo
 - Reconhecimento de dígitos: 10 dígitos.
 - Classificação de texto: Esportes, Economia, Política, Entretenimento, etc.
 - Classificação de sentimentos: Neutro, Positivo, Negativo.

Casos multiclases



- Quando $Q > 2$, chamamos o problema de classificação **multiclases**.
- Podemos resolver estes problemas com o **regressor *softmax***, que é uma generalização do regressor logístico.



Regressor *softmax*

- Também conhecida como **regressão logística multinomial**.
 - Pois suas saídas podem ser interpretadas como as **probabilidades de uma variável categórica** (as classes) **dado os atributos e pesos**.
 - Ou seja, a regressão ***softmax*** estima a **probabilidade de um exemplo de entrada pertencer a cada uma das Q possíveis classes**.
- O regressor tem Q saídas, onde cada saída representa a **probabilidade** de um exemplo pertencer a uma das Q classes.
 - Por exemplo, para um problema com 4 classes, teremos um classificador com 4 saídas.

Regressor *softmax*

- Ele prediz **apenas uma classe por classificação**, portanto, ele deve ser usado apenas com **classes mutuamente exclusivas** como, por exemplo, diferentes tipos de plantas, dígitos, carros, etc.
 - **Classes mutuamente exclusivas**: caso onde os exemplos pertencem a apenas uma das Q classes.
 - Já notícias, músicas e imagens, por exemplo, podem pertencer a várias ou conter várias classes ao mesmo tempo.
- O regressor *softmax* **possui uma função hipótese de classificação**, $h_a^q(\mathbf{x}(n))$, e, conseqüentemente, **uma função discriminante**, $g_q(\mathbf{x}(n))$, **para cada classe** q .
 - Se temos 3 classes, teremos 3 funções discriminantes, $g_q(\mathbf{x}(n))$.

Regressor *softmax*

- A **função hipótese de classificação** associada à q -ésima classe, $h_a^q(\mathbf{x}(n))$, é obtida passando-se a **função discriminante** da q -ésima classe, $g_q(\mathbf{x}(n))$, através da **função (de ativação) *softmax***,

$$P(C_q \mid \mathbf{x}(n), \mathbf{a}_q) = h_a^q(\mathbf{x}(n)) = \frac{e^{g_q(\mathbf{x}(n))}}{\sum_{j=0}^{Q-1} e^{g_j(\mathbf{x}(n))}} \in \mathbb{R} [0, 1],$$

onde $\mathbf{a}_q = [a_0^q \quad a_1^q \quad \cdots \quad a_K^q]^T \in \mathbb{R}^{K+1 \times 1}$ é o **vetor de pesos** da q -ésima **função discriminante**, $g_q(\mathbf{x}(n))$, e n indica o número da amostra.

- Cada função discriminante tem seu próprio vetor de pesos, $\mathbf{a}_q$.
- Além do regressor *softmax*, ela é usada na **camada de saída** de **redes neurais** empregadas em tarefas de **classificação**.

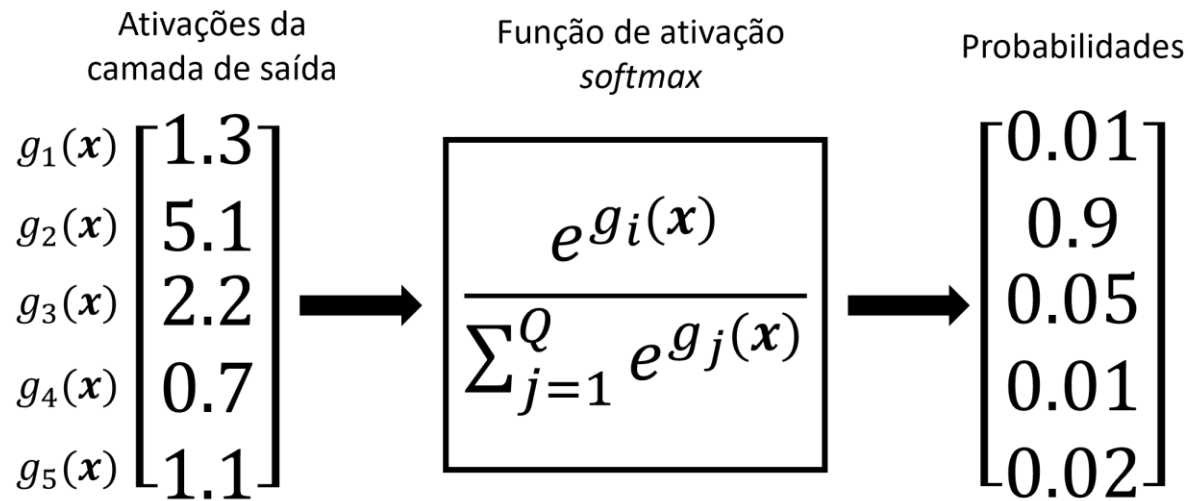
Função de ativação *softmax*

- Ela transforma um **vetor com valores de funções discriminantes em uma distribuição de probabilidades**.
 - O somatório de termos exponenciais no denominador **normaliza** o valor da q -ésima saída de tal forma que o somatório das Q saídas seja igual a 1

$$P(C_q \mid \mathbf{x}(n), \mathbf{a}_q) = h_a^q(\mathbf{x}(n)) = \frac{e^{g_q(\mathbf{x}(n))}}{\sum_{j=0}^{Q-1} e^{g_j(\mathbf{x}(n))}} \in \mathbb{R} [0, 1] \forall q.$$

- Dado um exemplo de entrada, ela **atribui** uma **probabilidade para cada classe** possível.
- É uma generalização da função sigmoide.

Função de ativação *softmax*



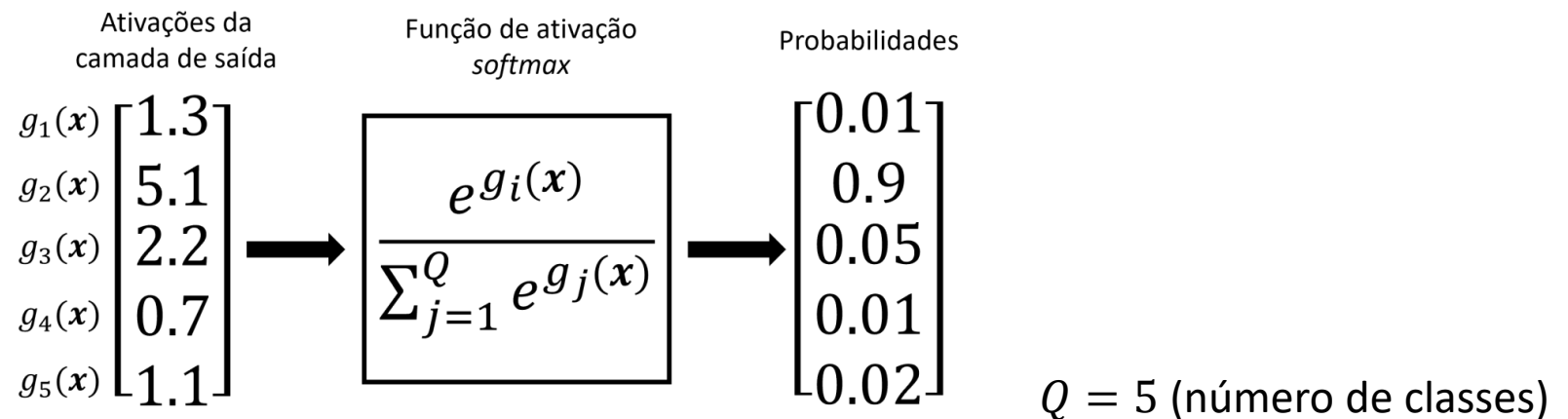
$Q = 5$ (número de classes)

- A função *softmax* opera da seguinte maneira.
- Dado um **vetor com valores de funções discriminantes**, $g_q(x(n)), \forall q$, ela
 - calcula a exponencial de cada função discriminante,
 - normaliza essas exponenciais dividindo-as pela soma de todas as exponenciais, gerando uma distribuição de probabilidades das classes.

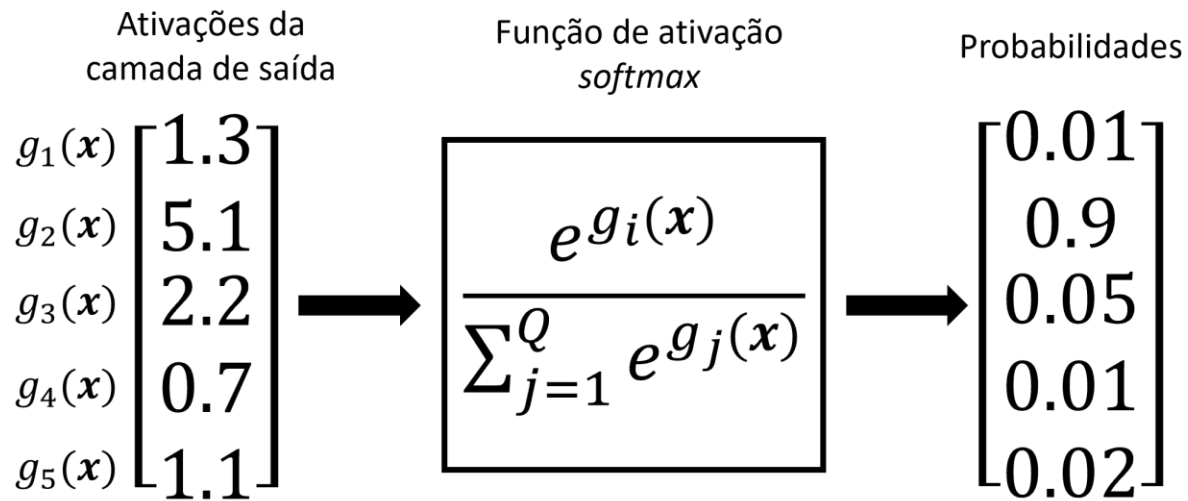
Função de ativação *softmax*

- Ela **garante que as saídas estejam no intervalo entre 0 e 1, e a soma de todas as saídas seja igual a 1**, tornando-as interpretáveis como uma **função massa de probabilidade multinomial**.
- Ela atribui uma **probabilidade condicional**, $P(C_q | \mathbf{x}(n), \mathbf{a}_q) \in [0, 1]$, a cada classe, q , onde **a soma das Q probabilidades deve ser igual a 1**

$$P(C_0 | \mathbf{x}(n), \mathbf{a}_0) + P(C_1 | \mathbf{x}(n), \mathbf{a}_1) + \cdots + P(C_{Q-1} | \mathbf{x}(n), \mathbf{a}_{Q-1}) = 1.$$



Função de ativação *softmax*



- Assim como com o regressor logístico, podemos usar **hiperplanos** ou **polinômios** como **funções discriminantes**.
- Nosso objetivo é encontrar um **modelo** (i.e., **equação** e os respectivos **pesos** das Q funções hipótese) que **atribua uma alta probabilidade para a classe alvo** e, consequentemente, **uma baixa probabilidade para as demais classes**.

Função de erro

- Para atingir o objetivo, devemos definir uma **função de erro** a ser **minimizada**.
- Usamos a generalização da função da entropia cruzada, chamada de **entropia cruzada categórica**, a qual é dada por

$$J_e(\mathbf{A}) = -\frac{1}{N} \sum_{n=0}^{N-1} \sum_{q=0}^{Q-1} 1\{y(n) == q\} \log(\hat{y}(n)),$$

onde $1\{\cdot\}$ é a **função indicadora** ($1\{\text{condição verdadeira}\} = 1$ e $1\{\text{condição falsa}\} = 0$), $\mathbf{A} \in \mathbb{R}^{K+1 \times Q}$ é a matriz com os **pesos** das Q **funções hipótese**, $y(n)$ é rótulo do n -ésimo exemplo e $\hat{y}(n) = h_a^q(\mathbf{x}(n))$ é a q -ésima saída do modelo para o n -ésimo exemplo.

- O erro tende a 0 quando $h_a^q(\mathbf{x}(n))$ tende a 1, caso contrário, o erro aumenta.

Função de erro

$$J_e(\mathbf{A}) = -\frac{1}{N} \sum_{n=0}^{N-1} \sum_{q=0}^{Q-1} 1\{y(n) == q\} \log(\hat{y}(n))$$

- A matriz $\mathbf{A}$ contém em suas colunas os vetores de pesos, $\mathbf{a}_q$, de cada uma das Q **funções discriminantes**.
- A função mede a **diferença entre as distribuições de probabilidade preditas pelo modelo e as distribuições dos rótulos** das classes.

Função de erro

- Usando a codificação **one-hot**, a equação pode ser re-escrita como

$$J_e(\mathbf{A}) = -\frac{1}{N} \sum_{n=0}^{N-1} \mathbf{y}(n)^T \log(\hat{\mathbf{y}}(n)),$$

onde $\mathbf{y}(n) = [1\{y(n) == 0\}, \dots, 1\{y(n) == Q - 1\}]^T \in \mathbb{R}^{Q \times 1}$ é um vetor com os rótulos usando a codificação **one-hot** e $\hat{\mathbf{y}}(n) = [\hat{y}_0(n), \dots, \hat{y}_{Q-1}(n)]^T \in \mathbb{R}^{Q \times 1}$ é um vetor com as Q saídas do modelo para o n -ésimo exemplo de entrada.

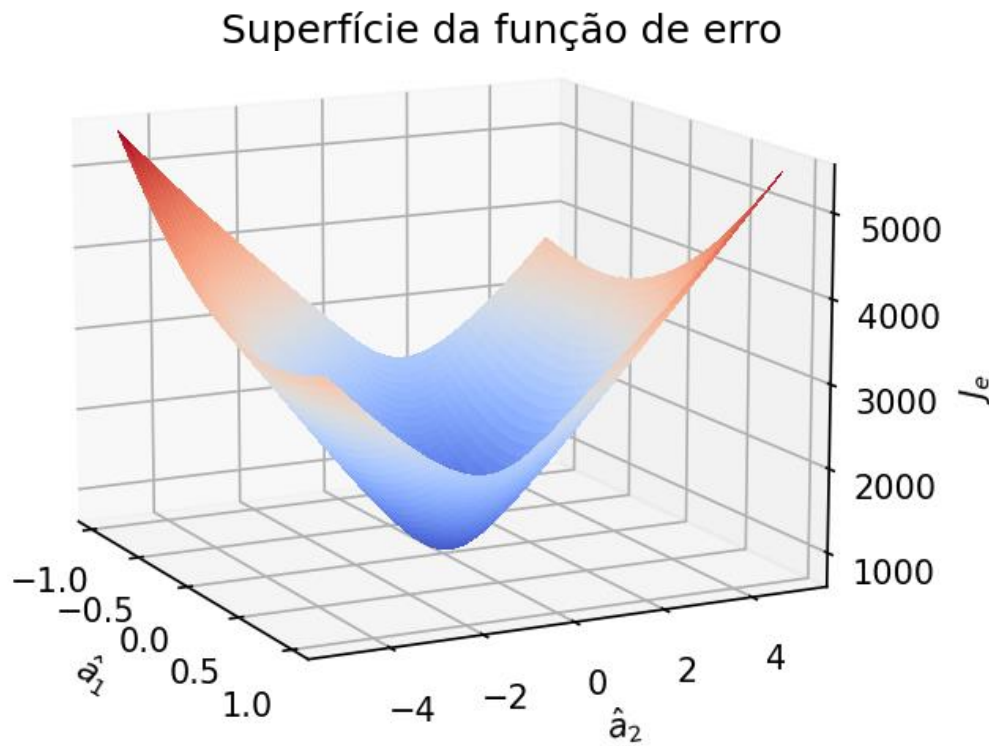
- Durante o **treinamento** do modelo, o **objetivo é minimizar a entropia cruzada categórica**, ou seja, fazer com que as **probabilidades previstas pelo modelo se aproximem ao máximo dos rótulos dos exemplos**.

Função de erro

$$J_e(\mathbf{A}) = -\frac{1}{N} \sum_{n=0}^{N-1} \mathbf{y}(n)^T \log(\hat{\mathbf{y}}(n)).$$

- Notem que quando existem apenas duas classes ($Q = 2$), a **função de erro médio** é equivalente à **função de erro médio** do **regressor logístico**.
- Ou seja, mesmo tendo sido pensado para o caso onde $Q > 2$, o regressor *softmax* também pode ser usado quando $Q = 2$.
- Entretanto, existe um problema com essa função de erro.

Função de erro



- Essa **função de erro não é linear** e, portanto, **não existe uma forma fechada** para encontrarmos os pesos.
- Porém, ela é **convexa** e **contínua**.
- Assim, **o algoritmo do gradiente descendente (qualquer versão) pode ser usado para encontrar o mínimo global**.

Gradiente descendente

- A atualização dos **pesos** da q -ésima função discriminante, $g_q(\mathbf{x}(n))$, é dada por

$$\mathbf{a}_q = \mathbf{a}_q - \alpha \frac{\partial J_e(\mathbf{A})}{\partial \mathbf{a}_q}, \forall q.$$

- Considerando o **hiperplano** como **função discriminante**, a derivada da **função de erro**, $J_e(\mathbf{A})$, com respeito ao vetor de pesos, $\mathbf{a}_q$, é dada por

$$\frac{\partial J_e(\mathbf{A})}{\partial \mathbf{a}_q} = -\frac{1}{N} \sum_{n=0}^{N-1} [1\{y(n) == q\} - h_a^q(\mathbf{x}(n))] \mathbf{x}(n) = -\frac{1}{N} \mathbf{X}^T (\mathbf{y}_q - \hat{\mathbf{y}}_q),$$

onde $\mathbf{y}_q$ e $\hat{\mathbf{y}}_q \in \mathbb{R}^{N \times 1}$ são vetores contendo os q -ésimos elementos dos vetores *one-hot* e de saída do regressor *softmax* para cada um dos N vetores de atributo, respectivamente.

Regressor *softmax*

$$\frac{\partial J_e(\mathbf{A})}{\partial \mathbf{a}_q} = -\frac{1}{N} \mathbf{X}^T (\mathbf{y}_q - \hat{\mathbf{y}}_q).$$

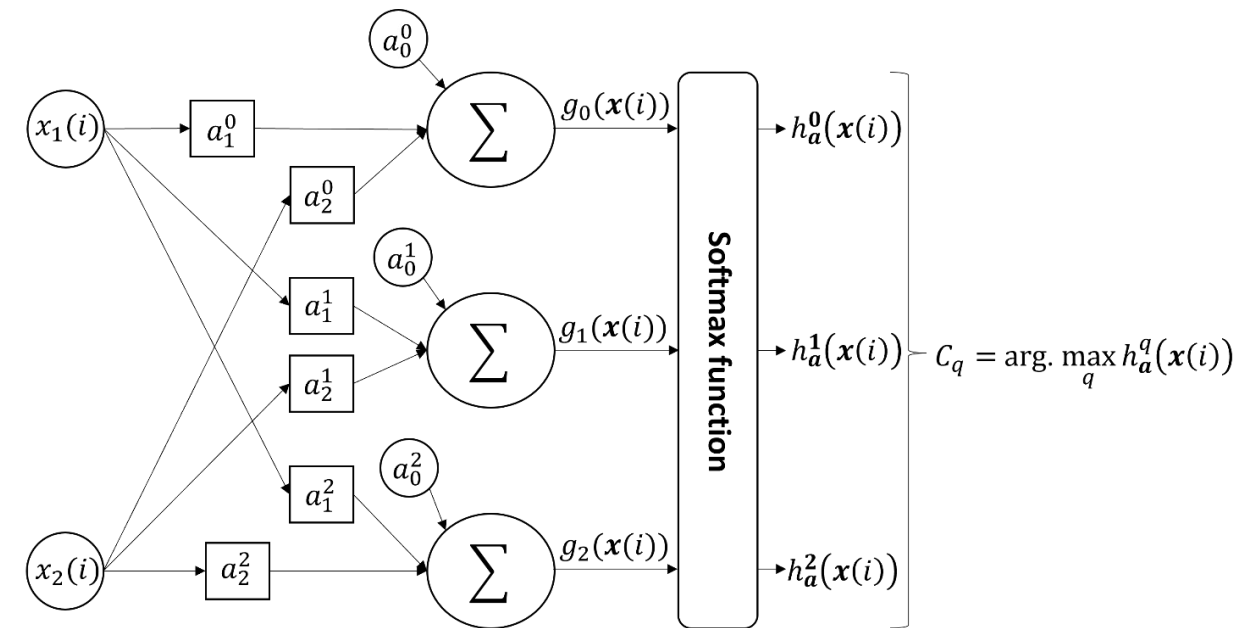
- Notem que essa expressão do vetor gradiente é **idêntica** àquela obtida para o caso da **regressão logística**.
- O vetor gradiente pode ser reescrito levando-se em consideração todos os Q vetores de peso

$$\frac{\partial J_e(\mathbf{A})}{\partial \mathbf{A}} = -\frac{1}{N} \mathbf{X}^T (\mathbf{Y} - \hat{\mathbf{Y}}),$$

onde $\mathbf{Y}$ e $\hat{\mathbf{Y}} \in \mathbb{R}^{N \times Q}$ são matrizes contendo em cada linha os vetores $\mathbf{y}(n)^T$ e $\hat{\mathbf{y}}(n)^T$ para todos os N exemplos, respectivamente.

- Esse resultado é diretamente estendido para polinômios.

Regressor *softmax*

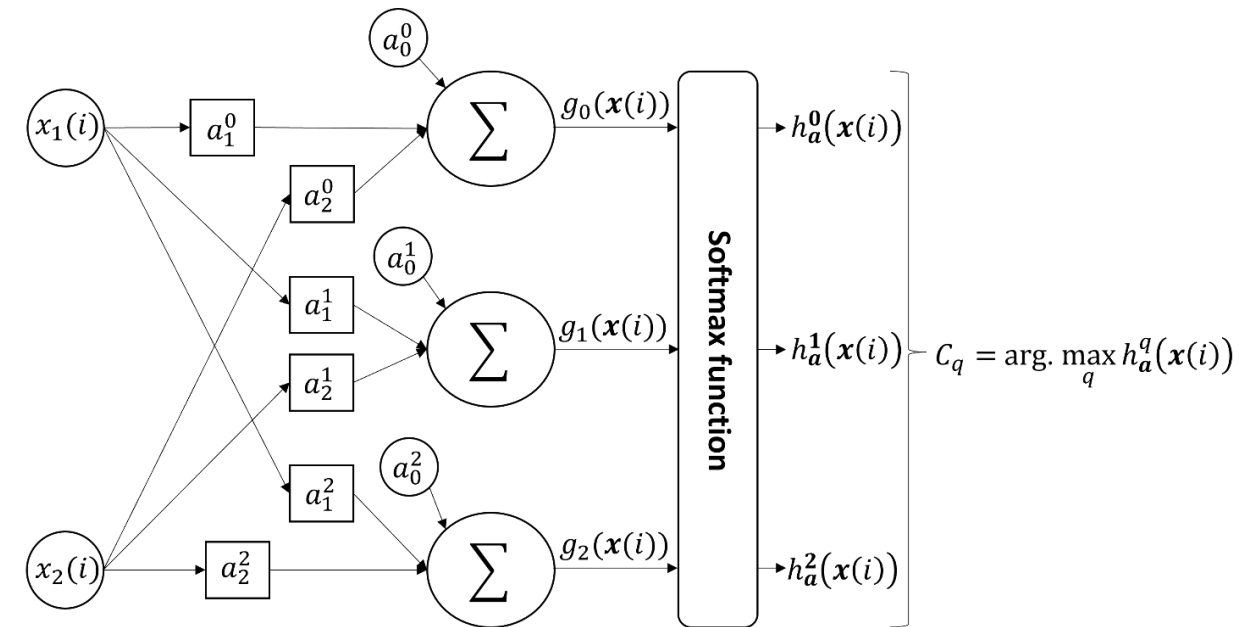


- Após o treinamento, durante a **fase de inferência**, o classificador atribui ao exemplo de entrada, $\mathbf{x}(n)$, a classe, q , com a **maior probabilidade estimada**, ou seja, atribui a classe, q , da função hipótese com maior valor:

$$C_q = \arg. \max_q h_a^q(\mathbf{x}(n)).$$

- A arquitetura de um **regressor softmax** para um problema com três classes (i.e., $Q = 3$) e dois atributos (x_1 e x_2) é mostrada ao lado.

Regressor *softmax*



- Em resumo, a ideia por trás da **regressão *softmax*** é bastante simples:
 - dado um exemplo de entrada, $\mathbf{x}(n)$,
 - o regressor *softmax* calcula uma “**pontuação**”, para cada classe q , i.e., as saídas das Q funções discriminantes $g_q(\mathbf{x}(n))$, $\forall q$.
 - e, em seguida, estima a probabilidade de cada classe aplicando a função *softmax* às “**pontuações**”.

Exemplo

- [Exemplo: regressão softmax.ipynb](#)



Lista de exercícios

Lista #7 – Regressão *softmax*

Perguntas?

Obrigado!